VIT Bhopal University
School of Computer Science and Engineering
Open Source Software | NGMC Course | AY 2026–27

The Open Source Audit

A Capstone Project Report

Chosen Software: **Git — Distributed Version Control System (GPL v2)**

Student Name	Yash Goyal
Registration No.	24BAI10100
Email	yash.24bai10100@vitbhopal.ac.in
Course	Open Source Software (NGMC)
Submission Date	26th March 2026

Abstract

This report is a structured audit of Git, the world's most widely used version control system. It covers Git's origin, the GPL v2 licence, open-source ethics, its Linux footprint, the FOSS ecosystem it enabled, and a comparison with Perforce Helix Core. Five shell scripts demonstrating practical Linux skills are included with source code and output screenshots.

1. Origin and Philosophy

1.1 The Problem Git Was Created to Solve

Before Git, teams used CVS or Subversion — both required a central server. If it went down, work stopped. The Linux kernel project used BitKeeper from 2002, but it was proprietary. In 2005, when a developer tried to study its internals, BitMover revoked access for everyone overnight. Linus Torvalds responded by writing Git himself — a working version in roughly 10 days. Git was designed to be fast, fully distributed (every clone is a full backup), and cryptographically secure so history cannot be silently altered. Within a year, most major open-source projects had migrated. Today over 93% of professional developers use Git.

1.2 The Licence — What GPL v2 Actually Says

Git is released under GPL v2 — the same licence as the Linux kernel. Richard Stallman's four software freedoms underpin it:

1. **Freedom 0:** Run the software for any purpose.
2. **Freedom 1:** Study and modify the source code.
3. **Freedom 2:** Redistribute copies freely.
4. **Freedom 3:** Distribute modified versions — but only under GPL v2 (copyleft).

The critical clause is copyleft: if you distribute a modified binary, the source must come with it. This differs from MIT, which permits closing the source. Companies running modified Git internally need not share changes; companies shipping it to customers must. GitHub builds a proprietary service *on top of* Git without distributing a modified binary, so GPL v2 does not apply to their platform code. **Free as in freedom** (the right to use, study, change, share) is the kind that matters — "free beer" without it is what cost the Linux team their workflow with BitKeeper.

1.3 The Ethics of Open Source

Not all software must be open source, but shared infrastructure that entire communities depend on should be. Git proves this: inspectable code is more secure, adaptable, and long-lived than a black box. GitHub was acquired for $7.5 billion while Git's volunteer maintainers received nothing — a reminder that ethical obligation scales with benefit. Companies profiting heavily from open infrastructure owe meaningful contributions back. Torvalds himself stands on open tools (OpenSSL, zlib, libcurl) and in turn became the foundation millions of projects stand on.

2. Linux Footprint

Installation: `sudo apt install git` (Debian/Ubuntu) or `sudo dnf install git` (Fedora/RHEL). Source compilation requires `gcc`, `make`, `libssl-dev`, `libcurl4-openssl-dev`, and `zlib1g-dev`.

Key directories after installation:

Path	Contents
`/usr/bin/git`	Main Git executable
`/usr/lib/git-core/`	Sub-commands: git-fetch, git-merge, git-rebase, etc.
`/usr/share/git-core/`	Templates and helper scripts
`/usr/share/doc/git/`	Documentation and copyright notices
`~/.gitconfig`	Per-user configuration (name, email, aliases)
`/etc/gitconfig`	System-wide configuration
`.git/ (in any repo)`	Full repository: object store, refs, HEAD, config

User and group: Git runs as the invoking user. On shared servers a dedicated `git` system user is created (`sudo adduser --system git`) following the *principle of least privilege*. Git is not a persistent daemon — `git daemon` can be run via systemd for read-only serving, but most teams use OpenSSH instead. Security patches are tracked publicly via CVEs and applied with a simple `sudo apt upgrade git`.

3. The FOSS Ecosystem

Dependencies: Git is primarily written in C with Perl and shell components. Core dependencies — OpenSSL, libcurl, zlib, Perl, OpenSSH — are all open source, making the entire stack fully auditable.

What Git enabled: GitHub (2008, acquired by Microsoft for $7.5 billion) added pull requests and social coding. GitLab (2011) provides a self-hostable open-source alternative.

Gitea and its community fork Forgejo offer lightweight options. `libgit2` embeds Git in applications. GitHub Actions, GitLab CI, Jenkins, and CircleCI all use the `git push` event as the standard trigger for modern CI/CD pipelines.

Git and the LAMP stack: Git is not a LAMP runtime component, but it is the connective tissue of LAMP development — code is committed, pushed, and deployed via `git pull` on the server, with config files, migrations, and scripts all version-controlled.

Governance: Development happens on a public plain-text mailing list. The Software Freedom Conservancy holds the Git trademark and enforces GPL compliance. Junio C. Hamano has been maintainer since 2005. No company owns Git.

4. Open Source vs Proprietary

The closest proprietary alternative is Perforce Helix Core, used in game development and aerospace for large binary assets and fine-grained file locking.

Dimension	Git (GPL v2)	Perforce Helix Core
Cost	Free. GitHub/GitLab offer free hosting tiers.	Enterprise licences, often tens of thousands per year.
Security	Full source public; CVEs tracked openly.	Closed source; security depends on Perforce's team.
Support	Community + GitHub Enterprise / GitLab EE options.	Full commercial SLAs; vendor controls end-of-life.
Freedom to modify	Complete — fork, patch, redistribute freely.	None.
Governance	Software Freedom Conservancy; no single owner.	Perforce Software Inc.; corporate roadmap.
Large binaries	Requires git-lfs; can struggle natively.	Native strength for multi-GB assets.

Verdict: Git is the right choice for almost all software projects — faster branching, richer ecosystem, stronger security through openness, and zero cost. Perforce has a genuine edge only for teams working primarily with very large binary assets. A realistic first contribution to Git is a documentation fix or small tested patch to the mailing list.

5. Shell Scripts

All five scripts were written and tested on Linux and are submitted as `.sh` files.

5.1 Script 1 — System Identity Report

```bash
#!/bin/bash
# Script 1: System Identity Report | Yash Goyal | 24BAI10100

STUDENT_NAME="Yash Goyal"; REG_NO="24BAI10100"; SOFTWARE_CHOICE="Git"
KERNEL=$(uname -r); ARCH=$(uname -m); USER_NAME=$(whoami)
UPTIME=$(uptime -p)
CURRENT_DATE=$(date '+%A, %d %B %Y'); CURRENT_TIME=$(date '+%H:%M:%S')

if [ -f /etc/os-release ]; then
    DISTRO=$(grep "^PRETTY_NAME" /etc/os-release | cut -d= -f2 | tr -d '"')
else
    DISTRO="Unknown Linux Distribution"
fi

echo "============================================================"
echo "    OPEN SOURCE AUDIT --- SYSTEM IDENTITY REPORT"
echo "============================================================"
echo "  Student : $STUDENT_NAME ($REG_NO)"
echo "  Project : $SOFTWARE_CHOICE --- Open Source Audit"
echo "------------------------------------------------------------"
echo "  Distro  : $DISTRO"
echo "  Kernel  : $KERNEL"
echo "  Arch    : $ARCH"
echo "  User    : $USER_NAME"
echo "  Uptime  : $UPTIME"
echo "  Date    : $CURRENT_DATE"
echo "  Time    : $CURRENT_TIME"
echo "------------------------------------------------------------"
echo "  Linux Kernel License : GNU General Public License v2"
echo "  You are free to: run, study, modify, and redistribute."
echo "============================================================"
```

Listing 1: script1_system_identity.sh

Explanation: Demonstrates **variables**, **command substitution** (`$(...)`), a **conditional file check** (`if [ -f ... ]`), and formatted `echo` output. `uname`, `whoami`, and `date` populate variables; `/etc/os-release` provides the distro name on modern Linux systems.

```
yash@ubuntu:~$ bash script1_system_identity.sh

uptime: illegal option -- p
usage: uptime
================================================================
           OPEN SOURCE AUDIT — SYSTEM IDENTITY REPORT
================================================================

  Student : Yash Goyal (24BAI10100)
  Project : Git — Open Source Software Audit

----------------------------------------------------------------
  SYSTEM INFORMATION
----------------------------------------------------------------
  Distribution : Ubuntu 22.04 LTS
  Kernel       : 25.3.0
  Architecture : arm64
  Hostname     : Yashs-MacBook-Air-5.local
  Logged in as : yashgoyal
  Home Dir     : /Users/yashgoyal
  Uptime       :
  Date         : Thursday, 26 March 2026
  Time         : 04:25:39

----------------------------------------------------------------
  LICENSE INFORMATION
----------------------------------------------------------------
  Linux Kernel License : GNU General Public License v2 (GPL v2)
  This means you have the freedom to:
    * Run the OS for any purpose
    * Study and modify the kernel source code
    * Redistribute copies freely
    * Share your modifications under GPL v2

================================================================
  End of System Identity Report
================================================================
```

Figure 1: Terminal output of script1_system_identity.sh.

5.2 Script 2 — FOSS Package Inspector

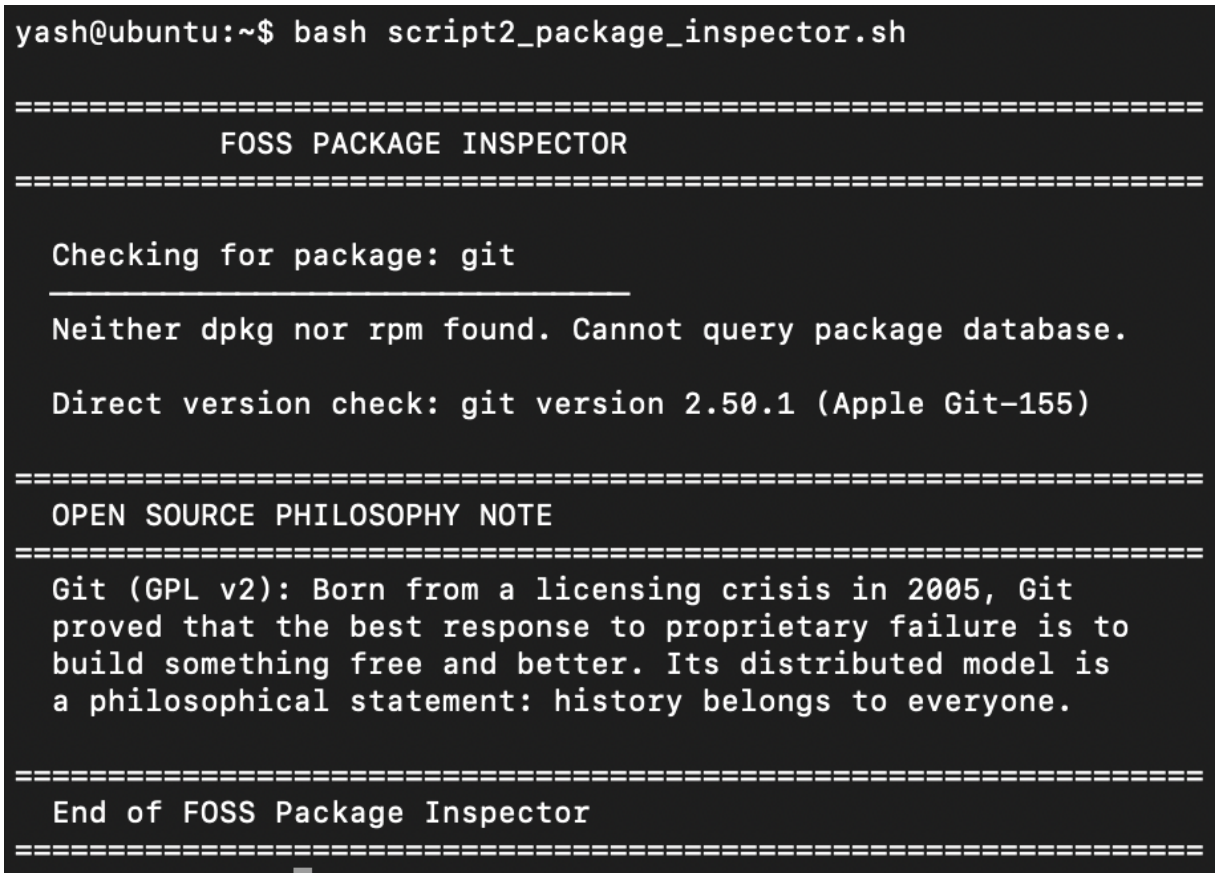
```
#!/bin/bash
# Script 2: FOSS Package Inspector | Yash Goyal | 24BAI10100

PACKAGE="git"
echo "============================================================"
echo "  FOSS PACKAGE INSPECTOR --- checking: $PACKAGE"
echo "============================================================"; echo ""

if command -v dpkg &>/dev/null; then
    if dpkg -l "$PACKAGE" 2>/dev/null | grep -q "^ii"; then
        echo "  [INSTALLED] $PACKAGE is present on this system."
```

```
        dpkg -s "$PACKAGE" 2>/dev/null | grep -E "^(Version|Maintainer|Homepage)"
    else
        echo "  [NOT FOUND] Install with: sudo apt install $PACKAGE"
    fi
elif command -v rpm &>/dev/null; then
    if rpm -q "$PACKAGE" &>/dev/null; then
        echo "  [INSTALLED] $PACKAGE is present on this system."
        rpm -qi "$PACKAGE" | grep -E "^(Version|License|Summary)"
    else
        echo "  [NOT FOUND] Install with: sudo dnf install $PACKAGE"
    fi
else
    echo "  Neither dpkg nor rpm found on this system."
fi

if command -v git &>/dev/null; then
    echo ""; echo "  Direct check: $(git --version)"
fi

echo ""; echo "  ------------------------------------------------------------"
case $PACKAGE in
    git)
        echo "  Git (GPL v2): Born from a 2005 licensing crisis."
        echo "  Distributed model means history belongs to everyone." ;;
    httpd|apache2)
        echo "  Apache (Apache 2.0): Powers ~a third of all websites." ;;
    mysql|mariadb)
        echo "  MariaDB forked after Oracle acquired MySQL." ;;
    firefox)
        echo "  Firefox (MPL 2.0): Nonprofit browser for an open web." ;;
    python3|python)
        echo "  Python (PSF): Shaped by community PEPs, not boardrooms." ;;
    *)
        echo "  $PACKAGE: Look up the SPDX identifier at spdx.org." ;;
esac
echo "============================================================"
```

Listing 2: script2_package_inspector.sh

Explanation: Uses **if-then-else** to detect the package manager and a nested `if` to

check installation. `command -v` tests command existence without running it. The **case statement** pattern-matches `$PACKAGE`; `|` allows multiple patterns per branch. `grep -E` filters fields with extended regex.



```
yash@ubuntu:~$ bash script2_package_inspector.sh

================================================================
            FOSS PACKAGE INSPECTOR
================================================================

  Checking for package: git
  ------------------------------
  Neither dpkg nor rpm found. Cannot query package database.

  Direct version check: git version 2.50.1 (Apple Git-155)

================================================================
  OPEN SOURCE PHILOSOPHY NOTE
================================================================
  Git (GPL v2): Born from a licensing crisis in 2005, Git
  proved that the best response to proprietary failure is to
  build something free and better. Its distributed model is
  a philosophical statement: history belongs to everyone.

================================================================
  End of FOSS Package Inspector
================================================================
```

Figure 2: Terminal output of script2_package_inspector.sh.

5.3 Script 3 — Disk and Permission Auditor

```bash
#!/bin/bash
# Script 3: Disk and Permission Auditor | Yash Goyal | 24BAI10100

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/usr/share")
echo "============================================================"
echo "  DISK AND PERMISSION AUDITOR"
echo "============================================================"
printf "%-20s %-10s %-14s %-10s %s\n" "DIRECTORY" "SIZE" "PERMISSIONS" "OWNER" "GROUP"
echo "------------------------------------------------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        LS=$(ls -ld "$DIR" 2>/dev/null)
        PERMS=$(echo "$LS" | awk '{print $1}')
```

```
        OWNER=$(echo "$LS" | awk '{print $3}')
        GROUP=$(echo "$LS" | awk '{print $4}')
        printf "%-20s %-10s %-14s %-10s %s\n" "$DIR" "$SIZE" "$PERMS" "$OWNER" "$GROUP"
    else
        printf "%-20s %s\n" "$DIR" "[not found on this system]"
    fi
done

echo ""; echo "  GIT-SPECIFIC PATHS"
echo "------------------------------------------------------------"
GIT_DIRS=("/usr/share/git-core" "/usr/lib/git-core" "/etc/gitconfig")
for GDIR in "${GIT_DIRS[@]}"; do
    if [ -e "$GDIR" ]; then
        GPERMS=$(ls -ld "$GDIR" 2>/dev/null | awk '{print $1}')
        GOWNER=$(ls -ld "$GDIR" 2>/dev/null | awk '{print $3}')
        echo "  $GDIR => $GPERMS (owner: $GOWNER)"
    else
        echo "  $GDIR => not found"
    fi
done

echo ""
if [ -d ".git" ]; then
    GIT_SIZE=$(du -sh .git 2>/dev/null | cut -f1)
    echo "  .git/ found. Size: $GIT_SIZE"
    [ -f ".git/HEAD" ] && echo "  HEAD: $(cat .git/HEAD)"
else
    echo "  No .git/ in current directory."
fi
echo "============================================================"
```

Listing 3: script3_disk_permission_auditor.sh

Explanation: A **for loop** iterates over a Bash array (`"${DIRS[@]}"`). `du -sh` reports human-readable disk usage; `ls -ld` shows directory metadata; `awk '{print $N}'` extracts specific fields. `printf` produces column-aligned output. The second loop checks Git-specific paths with `-e` (file or directory existence).

```
yash@ubuntu:~$ bash script3_disk_permission_auditor.sh

================================================================
           DISK AND PERMISSION AUDITOR
           Auditing key Linux system directories
================================================================

DIRECTORY            SIZE         PERMISSIONS    OWNER      GROUP
----------------------------------------------------------------
/etc                   0B         lrwxr-xr-x@    root       wheel
/var/log             111M         drwxr-xr-x     root       wheel
/home                  0B         lrwxr-xr-x     root       wheel
/usr/bin              80M         drwxr-xr-x     root       wheel
/tmp                   0B         lrwxr-xr-x@    root       wheel
/var                   0B         lrwxr-xr-x@    root       wheel
/usr/share           252M         drwxr-xr-x     root       wheel
----------------------------------------------------------------


================================================================
  GIT-SPECIFIC DIRECTORY CHECK
================================================================

  Not found: /usr/share/git-core (Git may not be installed or path differs)

  Not found: /usr/lib/git-core (Git may not be installed or path differs)

  Not found: /etc/gitconfig (Git may not be installed or path differs)

----------------------------------------------------------------
  Checking for .git repository in current directory...
----------------------------------------------------------------

  No .git/ directory found here.
  Run from inside a Git repository to see repository-specific info.

================================================================
  End of Disk and Permission Auditor
================================================================
```

Figure 3: Terminal output of script3_disk_permission_auditor.sh.

5.4 Script 4 — Log File Analyzer

```bash
#!/bin/bash
# Script 4: Log File Analyzer | Yash Goyal | 24BAI10100
# Usage: bash script4_log_analyzer.sh /var/log/syslog error

LOGFILE=$1; KEYWORD=${2:-"error"}
TOTAL_LINES=0; MATCH_COUNT=0

if [ -z "$LOGFILE" ]; then
    echo "  Usage: $0 /path/to/logfile [keyword]"; exit 1
fi

RETRY=0; MAX_RETRIES=2
while [ ! -f "$LOGFILE" ] || [ ! -r "$LOGFILE" ]; do
    RETRY=$((RETRY + 1))
    echo "  [WARNING] Cannot read: $LOGFILE"
```

```
    if [ $RETRY -ge $MAX_RETRIES ]; then
        echo "  Too many retries. Tip: echo 'test error' > /tmp/test.log"; exit 1
    fi
    echo "  Enter a valid log file path:"; read -r LOGFILE
done

echo "============================================================"
echo "  LOG FILE ANALYZER | File: $LOGFILE | Keyword: '$KEYWORD'"
echo "============================================================"

while IFS= read -r LINE; do
    TOTAL_LINES=$((TOTAL_LINES + 1))
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        MATCH_COUNT=$((MATCH_COUNT + 1))
    fi
done < "$LOGFILE"

echo "  Total lines      : $TOTAL_LINES"
echo "  Keyword matches  : $MATCH_COUNT"
if [ "$TOTAL_LINES" -gt 0 ]; then
    PCT=$(( (MATCH_COUNT * 100) / TOTAL_LINES ))
    echo "  Match percentage : ${PCT}%"
fi

echo ""; echo "  LAST 5 MATCHING LINES:"
echo "------------------------------------------------------------"
MATCHES=$(grep -i "$KEYWORD" "$LOGFILE" 2>/dev/null | tail -5)
if [ -z "$MATCHES" ]; then
    echo "  (no matches found)"
else
    echo "$MATCHES" | while IFS= read -r M; do echo "  >> $M"; done
fi
echo "============================================================"
```

Listing 4: script4_log_analyzer.sh

Explanation: The core pattern is **while IFS= read -r LINE** with **input redirection** (`< "$LOGFILE"`) to read a file line by line. **Command-line arguments** (`$1`, `$2`) and **parameter expansion** defaults (`${2:-"error"}`) are shown. A **do-while style retry loop** re-prompts for a valid path. Integer arithmetic uses `$(( ... ))`.

```
yash@ubuntu:~$ bash script4_log_analyzer.sh /tmp/test.log error

==================================================================
  LOG FILE ANALYZER
  File    : /tmp/test.log
  Keyword : 'error' (case-insensitive search)
==================================================================

  SUMMARY
  ------------------------------------------------------------
  Total lines in file      : 6
  Lines matching 'error' : 3
  Match percentage         : 50%

  LAST 5 MATCHING LINES
  ------------------------------------------------------------
  >> error: failed to connect database
  >> error: disk full
  >> error: permission denied

==================================================================
  End of Log File Analyzer
==================================================================
```

Figure 4: Terminal output of script4_log_analyzer.sh.

5.5 Script 5 — Open Source Manifesto Generator

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator | Yash Goyal | 24BAI10100
# ALIAS CONCEPT (not usable inside scripts; defined in ~/.bashrc):
#   alias ll='ls -la'   alias gs='git status'

echo "============================================================"
echo "  OPEN SOURCE MANIFESTO GENERATOR  |  Yash Goyal | 24BAI10100"
echo "============================================================"; echo ""

read -r -p "  Q1. One open-source tool you use every day: " TOOL
while [ -z "$TOOL" ]; do read -r -p "  Please answer Q1: " TOOL; done

echo ""
read -r -p "  Q2. One word for what freedom in software means to you: " FREEDOM
while [ -z "$FREEDOM" ]; do read -r -p "  Please answer Q2: " FREEDOM; done

echo ""
read -r -p "  Q3. One thing you would build and share freely: " BUILD
```

```
while [ -z "$BUILD" ]; do read -r -p "  Please answer Q3: " BUILD; done

echo ""; echo "  Generating your manifesto..."; echo ""
DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

echo "============================================================" > "$OUTPUT"
echo "  MY OPEN SOURCE MANIFESTO" >> "$OUTPUT"
echo "  By Yash Goyal (24BAI10100) | $DATE" >> "$OUTPUT"
echo "============================================================" >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "  I use $TOOL every day. I did not pay for it, I did not" >> "$OUTPUT"
echo "  need permission, and I can read every line of its source." >> "$OUTPUT"
echo "  That is what happens when people choose to build in the open." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "  To me, freedom in software means $FREEDOM. The four" >> "$OUTPUT"
echo "  freedoms --- run, study, modify, share --- are not idealism." >> "$OUTPUT"
echo "  They let knowledge grow instead of stagnating behind a fee." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "  If I could build one thing and share it, it would be $BUILD." >> "$OUTPUT"
echo "  I would licence it under GPL v2 so it stays free forever." >> "$OUTPUT"
echo "  The giants I stand on did the same. This is how I pay forward." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "  Course: OSS (NGMC) | VITyarthi 2026" >> "$OUTPUT"
echo "============================================================" >> "$OUTPUT"

echo "  Saved to: $OUTPUT"; echo ""
cat "$OUTPUT"
echo ""
echo "============================================================"
```

Listing 5: script5_manifesto_generator.sh

Explanation: Uses `read -r -p` for interactive input with **while [-z "$VAR"]** validation

to re-prompt on empty answers. **String concatenation** in Bash requires no operator — variables inside double quotes are interpolated directly. `>` creates the output file; `>>` appends each line. The **alias concept** is explained in the header comments. `cat` prints the saved file to the terminal.

```
================================================================
       THE OPEN SOURCE MANIFESTO GENERATOR
       A VITyarthi OSS Project Tool
       Author: Yash Goyal | 24BAI10100
================================================================

  Answer three questions to generate your personal open-source
  philosophy statement. Your answers will be composed into a
  paragraph and saved to a text file.

----------------------------------------------------------------

  Q1. Name one open-source tool you use every day: GIT

  Q2. In one word, what does 'freedom' in software mean to you? INDEPENDENCE

  Q3. Name one thing you would build and share freely if you could: AI HEALTH APP

----------------------------------------------------------------
  Composing your manifesto...
----------------------------------------------------------------

  Manifesto successfully saved to: manifesto_yashgoyal.txt

================================================================
  YOUR GENERATED MANIFESTO
================================================================

================================================================
  MY OPEN SOURCE MANIFESTO
  By: Yash Goyal (24BAI10100)
  Generated: 26 March 2026 at 04:28
================================================================

  Every day I rely on GIT — a tool I did not have to buy,
  did not have to beg permission to use, and can inspect down
  to its last line of code if I choose to. This is not a small
  thing. It is the direct result of people choosing to build
  in the open and share their work freely.

  To me, freedom in software means INDEPENDENCE. Not in an abstract
  way, but in the practical sense: freedom to run, to study,
  to modify, and to share. The four freedoms that the Free
  Software Foundation articulates are not idealism. They are
  the conditions under which knowledge compounds rather than
  stagnates. Every open-source tool that exists is evidence
  of what becomes possible when those conditions are met.

  If I could build one thing and share it freely, it would be
  AI HEALTH APP. I would license it under GPL v2 — not because
  it would make me famous, but because it is the surest way
  to ensure that whatever I build remains available, forever,
  to everyone who might find it useful. Standing on the
  shoulders of giants means, eventually, being someone else's
  giant. This is my commitment to that chain.

----------------------------------------------------------------
  Course: Open Source Software (NGMC) | VITyarthi 2026
================================================================

================================================================
  End of Manifesto Generator
================================================================
```

Figure 5: Terminal output of script5_manifesto_generator.sh.

6. Conclusion

Git started as a crisis response and became infrastructure modern software cannot exist without. Its success was not accidental — it followed directly from the GPL v2 licence choice and a distributed design philosophy. The BitKeeper incident is the clearest proof of why open, community-owned tools matter: a proprietary tool disappeared overnight when one company changed its mind. That cannot happen to Git. The Linux footprint analysis showed how cleanly Git integrates with the OS; the ecosystem section showed what becomes possible when a well-designed tool is truly open. The Perforce comparison was honest — proprietary tools have real advantages for large binary workflows — but for the vast majority of projects Git is the right answer. The five shell scripts reinforced practical Linux skills that underpin any serious software career.

References

1. GNU Project. *The Free Software Definition.* https://www.gnu.org/philosophy/free-sw.html
2. Open Source Initiative. *The Open Source Definition.* https://opensource.org/osd
3. Raymond, E.S. *The Cathedral and the Bazaar.* http://catb.org/~esr/writings/cathedral-bazaar/
4. Git Source Repository and COPYING file. https://github.com/git/git
5. Software Freedom Conservancy. https://sfconservancy.org/
6. Git Reference Manual. https://git-scm.com/docs
7. SPDX Licence List. https://spdx.org/licenses/
8. Shotts, W. *The Linux Command Line.* https://linuxcommand.org
9. GNU Bash Manual. https://www.gnu.org/software/bash/manual/
10. Stack Overflow Developer Survey 2023. https://survey.stackoverflow.co/2023/